

AI Agent Architecture

AI Agent Architecture

1. What Is an AI Agent?

Most people are familiar with ChatGPT or similar tools where you type a question and get a text answer back. That is what a standard large language model (LLM) does. An AI agent is a step beyond that. Instead of just answering, an agent can take actions. It can run functions, search documents, do calculations, and keep working through a problem step by step until it has a complete answer.

Think of it this way: a standard LLM is like a very knowledgeable person who can only talk. An AI agent is like that same person but also able to pick up a phone, look something up in a filing cabinet, or do math on a calculator before answering.

For this project, the agent receives a hospital bill or insurance document from a patient, figures out what information it needs, calls the right tools to get that information, and then produces a plain language explanation with next steps. The loop that powers this behavior works as follows:

Step	What Happens	Example in This Project
1	Patient sends a message or uploads a document	Patient uploads a hospital bill as a PDF
2	The LLM reads the input and decides what to do	The agent identifies it is a facility bill with multiple line items
3	The LLM calls one or more tools	It calls the bill parser tool to extract line items from the PDF
4	Each tool runs and returns a result	The parser returns a structured list of codes, descriptions, and amounts
5	The LLM reviews the result and decides the next step	The agent calls the code translator tool on each unfamiliar code
6	This repeats until the agent has everything it needs	The agent also checks financial assistance eligibility before answering
7	The agent produces a final response for the patient	Patient receives a plain language explanation with talking points and next steps

Important: This entire loop is already built inside the file `agent_harness/core.py` and should not be modified. The team builds the tools and instructions that plug into it, not the loop itself.

2. Tool Calling and Function Calling

Tool calling is the specific mechanism that allows the agent to run a Python function during a conversation. Each tool is a regular Python function that is wrapped with a special decorator called `@tool`. This decorator tells the agent what the function is named, what it does in plain English, and what inputs it expects.

The agent reads the description of each available tool and decides which one to call based on what the patient needs. This is why writing a clear description for each tool is so important. A vague description causes the agent to call the wrong tool or skip a tool it should have used.

Tool Name	What It Does in Plain Terms	When the Agent Uses It	Status
explain_bill	Takes one line item from a bill and writes a plain English explanation of what it means	Patient asks what a specific charge on their bill is for	Starter version provided. Needs to be expanded to handle CPT codes, HCPCS codes, revenue codes, and DRG codes
calculate_fpl	Calculates what percentage of the Federal Poverty Level the patient falls at based on their income and household size	Patient mentions their income or asks whether they qualify for financial help	Starter version provided. Needs to be validated against 2026 federal poverty guidelines
search_knowledge_base	Searches through Cedars-Sinai policy documents to find relevant information about billing and financial assistance	Patient asks about hospital policies, payment programs, or definitions of billing terms	Starter version provided. RAG upgrade needed (covered in Section 4)
parse_bill_document	Reads an uploaded PDF or image of a hospital bill and pulls out the structured data (codes, amounts, dates, payer info)	Patient uploads a document at the start of the conversation	Must be built by the team. This is the most technically complex tool in the project
translate_code	Converts any billing code (CPT, HCPCS, ICD-10, or revenue code) into a plain English description	Any unrecognized code appears on a bill or EOB	Must be built. Should reference the Cedars-Sinai chargemaster and CMS code databases

generate_action_plan	Creates a set of specific talking points and next steps the patient can use when calling their billing department or insurance company	After the bill explanation is complete	Must be built. Output should be specific to the patient's situation, not generic advice
-----------------------------	--	--	---

Every new tool must be registered in `src/app/tools/__init__.py` to be visible to the agent. The `@tool` decorator handles the technical setup automatically.

3. Safety Hooks

A hook is a piece of code that runs automatically before or after every tool call. Hooks cannot be bypassed by the agent. They sit in between the agent loop and the tool execution layer and act as a mandatory checkpoint.

The purpose of hooks in this project is twofold: protecting patient privacy and preventing the agent from saying something it should not say. Two hooks are already provided in the starter code.

Hook Name	What It Does	Why It Is Required	File Location
PHI Redaction	Automatically strips out any patient identifiers (such as name, date of birth, address, account number, or Social Security number) from inputs and outputs before they are logged or returned to the user	HIPAA law requires that real patient data is never stored or transmitted in plain text. All data in this project is synthetic, but this hook must still be in place for production readiness.	<code>app/hooks/phi_redaction.py</code>
Content Filter	Blocks the agent from producing medical advice, legal guarantees, promises about billing outcomes, or any response that could create legal liability for Cedars-Sinai	The agent's role is to explain and guide, not to diagnose or make legal determinations. Cedars-Sinai cannot be exposed to liability from AI-generated clinical or legal statements.	<code>app/hooks/content_filter.py</code>

Additional Hooks the Team Should Consider Building

- Anomaly flag hook: marks suspicious patterns on a bill (such as duplicate charges or unexpected out-of-network fees) before the agent responds, so the patient is alerted

- Tone enforcement hook: reviews every response before it reaches the patient to confirm it uses plain language and an empathetic tone

4. RAG (Retrieval Augmented Generation)

RAG is the system that allows the agent to search real documents before answering. Without RAG, the agent would rely entirely on what the LLM already knows from its training data, which could be outdated, incomplete, or simply wrong for Cedars-Sinai specifically.

With RAG, when a patient asks about financial assistance eligibility, the agent searches the actual Cedars-Sinai Financial Assistance Policy PDF and uses what it finds there to answer. Every claim the agent makes can be traced back to a real source document.

The system works in two phases:

- Indexing phase (runs once at startup): all documents in the knowledge-docs folder are read, broken into smaller chunks, and stored in a searchable index
- Retrieval phase (runs every time a patient asks something): the patient's query is matched against the index and the most relevant chunks are pulled out and given to the LLM as context for its answer

The Difference Between TF-IDF and Embeddings

The starter code uses TF-IDF, which is a basic keyword matching approach. The upgrade target is embeddings, which is a smarter approach based on meaning rather than exact words. The table below explains the difference in plain terms.

	TF-IDF (Current Starter)	Embeddings (Upgrade Target)
How it works	Counts how often specific words appear in documents and ranks results by word frequency matches	Converts text into a numeric representation that captures the meaning of words. Finds results that mean the same thing even if different words are used.
Simple example	If the patient types charity care, it finds documents containing those exact words and nothing else	If the patient types I cannot afford this bill, it finds documents about financial hardship and assistance programs even though those exact words do not appear
Why it matters here	Patients do not speak in billing terminology. They will rarely use the exact phrases that appear in hospital policy documents.	Embeddings bridge the gap between how patients talk and how documents are written, which dramatically improves how useful the agent is

What needs to change	Nothing. This is already working in the starter code.	Add sentence-transformers and chromadb to requirements.txt and replace the search class in src/app/rag/search.py
-----------------------------	---	--

The repo README already outlines exactly how to make this upgrade. It is estimated to require approximately 30 to 50 lines of new code.

5. Prompt Engineering for Agent Behavior

Prompt engineering is the practice of writing clear, structured instructions that tell the AI how to behave. In this project, it does not mean typing questions into ChatGPT. It means writing the rules and procedures that the agent follows in every conversation with a patient.

This is one of the most important parts of the project. Research on AI in healthcare settings has shown that well-engineered prompts can improve response accuracy from around 80% to over 99%. Getting the prompts right is as important as writing the code.

Prompt engineering in this project operates through three mechanisms:

Mechanism	What It Is	What It Controls	Where to Find It
Skills	Markdown files that are automatically loaded into every conversation as part of the system prompt	Step by step instructions for how the agent should approach a task. For example, the order in which it should analyze a bill, what to check for, and how to handle edge cases.	src/app/skills/
Roles	Markdown identity documents that define who a sub-agent is and what it specializes in	The persona, area of expertise, and boundaries of a specialized sub-agent. A financial assistance specialist role, for example, would be focused only on eligibility and payment programs.	src/app/roles/

Tool Descriptions	The plain English description written inside each tool definition using the <code>@tool</code> decorator	When the agent decides to call a tool and what information it passes to it. A vague description leads to the agent skipping tools it should have used.	<code>src/app/tools/*.py</code>
--------------------------	--	--	---------------------------------

What a Good Skill Looks Like

A skill is a structured set of written instructions in a markdown file. It is not code. Anyone on the team can write or improve a skill. A well-written skill tells the agent exactly what to do, in what order, in what tone, and what to avoid. Below is an example structure for the bill analysis skill.

Section	What to Write
Goal	Help the patient understand every charge on their bill in plain language without using any technical jargon
Steps	Step 1: Identify the document type before explaining anything (facility bill, physician bill, EOB, or patient statement). Step 2: Translate every billing code into plain English. Step 3: Walk through the financial math so the patient understands what they actually owe. Step 4: Flag anything that looks unusual. Step 5: Check whether financial assistance is relevant. Step 6: Produce talking points for the patient to use when calling billing or insurance.
Tone	Always be empathetic and calm. Assume the patient has no background in medical billing. Use simple language that a sixth grader could understand.
Constraints	Never give medical advice. Never promise a billing outcome. Never guess the meaning of an unknown code. If a code cannot be verified, tell the patient to call the billing department for a plain language description.

6. Sub-Agents and Specialized Roles

Rather than using one general agent to handle everything, the harness supports spinning up multiple specialized sub-agents. Each sub-agent has a specific area of focus, its own role document, and its own set of tools. The main agent calls a sub-agent the same way it calls a regular tool.

This approach keeps each part of the system focused and prevents the agent from mixing up tasks. For example, explaining a line item and calculating financial assistance eligibility require very different knowledge and tone. Keeping them as separate agents makes both better.

The repo already provides two starter roles. Below are the additional sub-agents the team should build:

Sub-Agent	Area of Focus	Why It Should Be a Separate Agent
Financial Assistance Specialist	FPL calculation, charity care eligibility, payment plan options, and Cedars-Sinai FAP policy	Needs deep knowledge of Cedars-Sinai policy documents specifically. Keeping it separate prevents the bill explainer from confusing financial guidance with billing explanation.
Dispute and Appeals Agent	Identifying potential billing errors, explaining insurance denial codes, and drafting appeal language for the patient	Disputes require a different tone than general explanations. Explanation is empathetic and educational. Dispute guidance needs to be assertive and action-oriented.
Action Planner	Generating specific talking points and a step by step call script for reaching out to the billing department or insurance company	This is a downstream task that runs after the explanation is complete. Keeping it separate ensures the agent does not rush to action steps before fully explaining the bill.

7. Hallucination Prevention and Grounded Responses

Hallucination is when an AI generates information that sounds confident and correct but is actually made up. In most applications this is annoying. In a healthcare billing context it is a serious problem. A patient who follows incorrect guidance about financial assistance eligibility, appeal deadlines, or what they owe could face real financial harm.

Preventing hallucination is one of the core responsibilities of the AI Agent Architecture track. It is addressed through a combination of RAG (grounding answers in real documents), clear skill instructions (telling the agent when to admit it does not know something), and safety hooks (blocking overconfident outputs).

Risk Area	What Hallucination Looks Like	How to Prevent It
Billing Code Translation	The agent invents a plain English description for a CPT or HCPCS code it does not recognize	The translate_code tool must check the Cedars-Sinai chargemaster and CMS reference databases first. If the code is not found, the agent must say it cannot verify the code and direct the patient to call billing.

Financial Assistance Rules	The agent quotes FPL thresholds or charity care percentages that differ from actual Cedars-Sinai policy	All financial assistance logic must be grounded in the full-fap-english.pdf knowledge document. The agent should always reference the source when stating eligibility information.
Insurance Details	The agent describes how a specific insurance plan works based on general knowledge rather than what is actually shown on the patient's EOB	All insurance-related statements must be anchored to information extracted from the uploaded document. The agent must not supplement with general knowledge about insurance plans.
Dollar Amounts	The agent calculates or states a patient balance that does not match what is actually on the bill	Amounts must always be extracted directly from the document. If the agent cannot confirm a number from the source, it must flag the uncertainty rather than estimate.

The single most important prompt engineering decision in this project is telling the agent to say 'I do not know' rather than guessing. This instruction must appear explicitly in the skills and system prompt.

8. How Everything Connects to the Project Deliverables

The table below maps every architectural concept covered in this research to the specific files the team will build, what success looks like, and what goes wrong if it is not done correctly.

Concept	Files to Build or Modify	What Good Looks Like	What Goes Wrong Without It
Tool Calling	src/app/tools/ parse_bill.py, translate_code.py, generate_action_plan.py	The agent reliably calls the right tool with the correct inputs every time a patient uploads a bill	The agent answers billing questions from general knowledge instead of actually reading the uploaded document. Answers are generic and potentially wrong.

RAG Upgrade	src/app/rag/search.py and requirements.txt	Patient can use natural everyday language and the agent still finds the right policy information	The search misses relevant policy documents because the patient did not use exact billing terminology. The agent fills the gap with hallucinated information.
Skills and Roles	src/app/skills/*.md and src/app/roles/*.md	Every conversation follows a consistent structured approach and the agent never skips checking financial assistance eligibility	The agent behaves inconsistently. It may explain some bills well and others poorly. It may forget to mention financial assistance options for patients who need them.
Safety Hooks	src/app/hooks/phi_redaction.py and content_filter.py	No patient identifiers appear in logs. The agent never makes a medical or legal claim.	Patient data could be exposed in server logs. The agent could produce a statement that creates legal liability for Cedars-Sinai.
Grounding and Hallucination Prevention	System prompt and skill files with explicit uncertainty instructions	Every factual claim in the agent's response can be traced back to the uploaded bill or a Cedars-Sinai knowledge document	The agent states incorrect billing amounts, fabricated policy rules, or wrong eligibility determinations with high confidence. A patient acts on that information.

Key References

- Patient-Billing-Agent GitHub Repository: github.com/destuar/Patient-Billing-Agent
- Anthropic Tool Use Documentation: docs.anthropic.com/en/docs/build-with-claude/tool-use
- OpenAI Function Calling Guide: platform.openai.com/docs/guides/function-calling
- Sentence Transformers Library: [sbert.net](https://www.sbert.net)
- ChromaDB Vector Database: trychroma.com
- Anthropic Contextual Retrieval: anthropic.com/news/contextual-retrieval
- Prompt Engineering Guide for AI Agents: promptingguide.ai/agents/function-calling
- CMS CPT and HCPCS Code Reference: [cms.gov](https://www.cms.gov)